

Observer, Command e Iterator

Padrões de Projeto - Aula #09

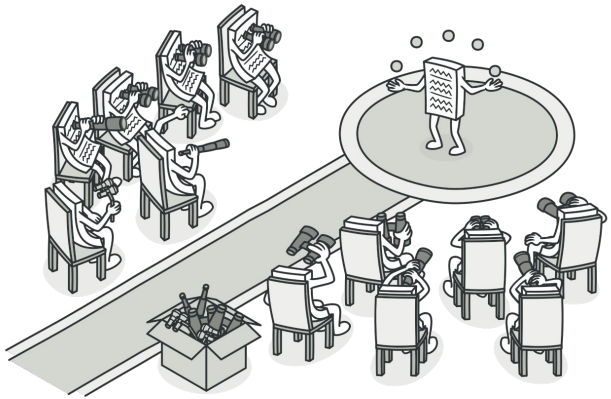
Prof. Dr. Carlos Melo

carlos.melo@upe.br

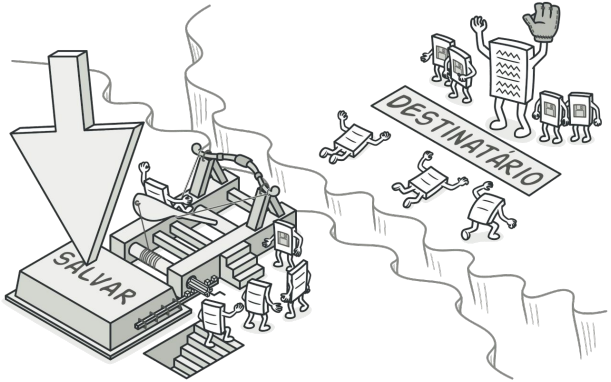
UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS

Padrões Comportamentais (parte #2)

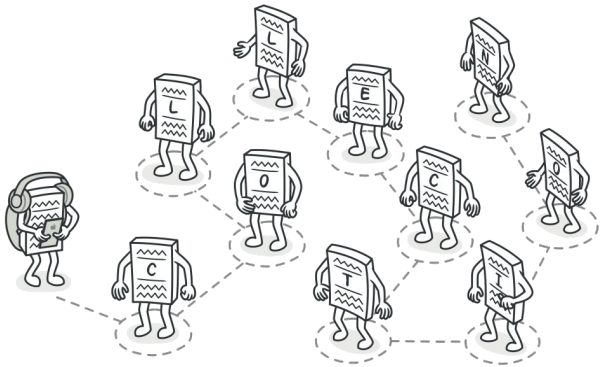
Observer
Permite que objetos se inscrevam para receber notificações sobre eventos de outro objeto



Command
Encapsula uma solicitação em um objeto que reúne os dados necessários para executá-la



Iterator
Percorrer uma coleção sem expor sua representação interna



- `Observer` organiza quem recebe uma notificação
- `Command` organiza como representar e executar uma solicitação
- `Iterator` diz como percorrer uma coleção

carlos.melo@upe.br

Objetivos de aprendizagem

- Implementar um mecanismo de inscrição, cancelamento e notificação usando `Observer`
- Adicionar novos observadores sem alterar a lógica do publicador
- Identificar os papéis de cliente, invocador, comando e receptor no `Command`
- Separar a configuração de uma solicitação do momento de sua execução
- Distinguir a responsabilidade da Coleção da responsabilidade do `Iterador`

Observer - O Problema

- Uma loja está aguardando a chegada de um novo modelo de iPhone
- Alguns clientes querem saber assim que o produto estiver disponível
- Uma possibilidade é cada cliente consultar a loja repetidamente
  - A maioria das consultas será inútil enquanto o produto não chegar
- Outra possibilidade é a loja avisar todos os clientes
  - Muitos receberão uma mensagem sobre um produto pelo qual não têm interesse
- Como avisar somente os interessados, no momento em que o produto chegar?



Observer - A ideia

- A loja mantém uma lista de clientes que solicitaram o aviso
  - O cliente se inscreve para receber a notificação
  - A loja recebe o produto e notifica os inscritos
  - Cada inscrito decide como reagir ao aviso
  - O cliente pode cancelar sua inscrição
- Os participantes têm dois papéis:
  - Publicador, ou `Subject`: mantém as inscrições e emite as notificações
  - Observador, ou `Observer`: recebe a notificação por uma interface comum
- A loja chama a mesma operação de atualização para todos os inscritos
- Cada observador implementa sua própria reação

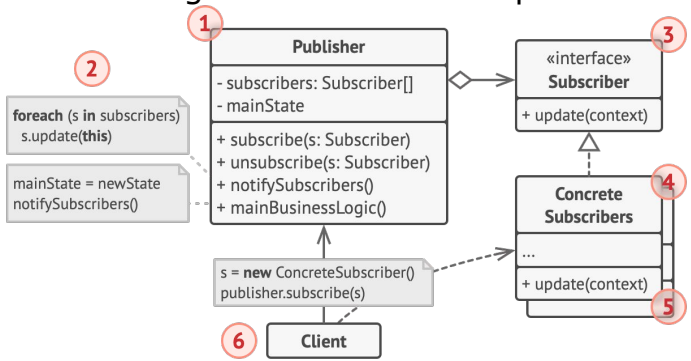
Observer - Intenção

"Definir uma relação um-para-muitos: quando um objeto muda, os objetos interessados recebem uma notificação"

- O publicador mantém os observadores inscritos
- Todos os observadores oferecem uma operação comum de atualização
- O publicador utiliza essa interface para notificá-los
- Novos tipos de observadores podem participar sem alterar a lógica de notificação

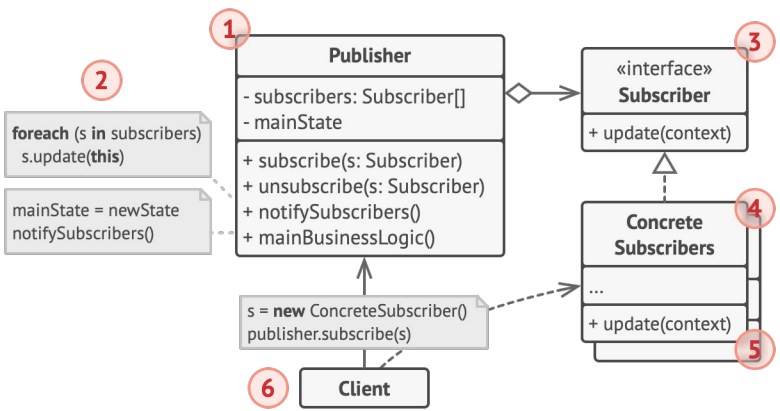
Observer - Estrutura

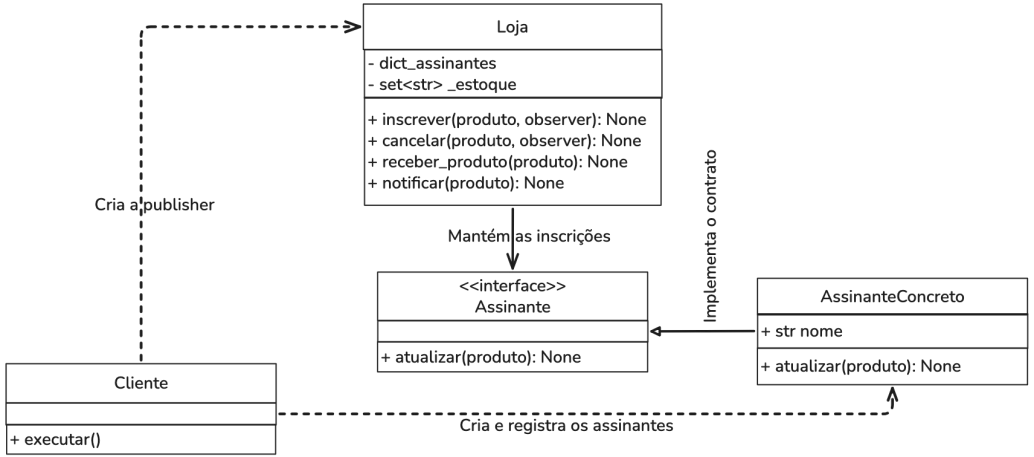
1. O **Publisher** manda eventos de interesse para outros objetos
   a. Eventos ocorrem quando o `publisher` muda seu estado ou executa algum comportamento
   b. Publishers contêm uma infraestrutura de inscrição que permite novos assinantes se juntar aos atuais *subscribers* ou deixar a lista
2. Quando um novo evento acontece, o `publisher` percorre a lista de assinantes e chama o método de notificação declarado na interface do `subscriber` em cada objeto assinante
3. A interface do **Subscriber** declara a interface de notificação
4. **ConcreteSubscribers** realizam algumas ações em resposta às notificações enviadas pelo publisher.
5. Publishers quase sempre passam algum dado de contexto como argumentos do método de notificação.
6. O **Client** cria a publisher e os objetos subscribers separadamente e então registra os assinantes para as atualizações da publicadora.



carlos.melo@upe.br

Aplicando ao nosso contexto





Aplicando ao nosso contexto

```python
from abc import ABC, abstractmethod


class Assinante(ABC):
    @abstractmethod
    def atualizar(self, produto: str) -> None:
        pass
```

```python
class AssinanteConcreto(Assinante):
    def __init__(self, nome: str) -> None:
        self.nome = nome

    def atualizar(self, produto: str) -> None:
        print(f"{self.nome}: recebi o aviso de que {produto} chegou!")
```

```python
class Loja:
    def __init__(self) -> None:
        self._estoque: set[str] = set()
        self._assinantes: dict[str, list[Assinante]] = {}

    def inscrever(self, produto: str, assinante: Assinante) -> None:
        interessados = self._assinantes.setdefault(produto, [])
        if assinante not in interessados:
            interessados.append(assinante)

    def cancelar(self, produto: str, assinante: Assinante) -> None:
        self._assinantes[produto].remove(assinante)

    def receber_produto(self, produto: str) -> None:
        self._estoque.add(produto)
        print(f"Loja: recebemos {produto}.")
        self.notificar(produto)

    def notificar(self, produto: str) -> None:
        for assinante in self._assinantes.get(produto, []):
            assinante.atualizar(produto)
```

```python
class Cliente:
    @staticmethod
    def executar() -> None:
        loja = Loja()
        ana = AssinanteConcreto("Ana")
        bruno = AssinanteConcreto("Bruno")

        loja.inscrever("iPhone", ana)
        loja.inscrever("iPhone", bruno)

        loja.receber_produto("iPhone")

        loja.cancelar("iPhone", ana)
        loja.receber_produto("iPhone")

        loja.receber_produto("Notebook")
```

carlos.melo@up

Prós e contras do Observer

Prós

- Novos assinantes concretos podem participar sem alterar a lógica de notificação do Publisher
- As inscrições podem mudar durante a execução
- Cada assinante concentra sua própria reação ao evento

Contras

- O fluxo de execução passa por vários objetos, o que pode dificultar o acompanhamento
- Na nossa implementação, as notificações são síncronas: a loja aguarda cada chamada de atualizar() terminar
- Uma exceção em um assinante interrompe o laço de notificação, caso não seja tratada

Use:

- Quando o conjunto de interessados pode variar e a publicadora precisa se comunicar com eles por um contrato comum

Command - O Problema

- Estamos desenvolvendo um editor de texto
- A operação de copiar pode ser acionada por diferentes elementos:

Origem da solicitação	Operação
Botão da barra de ferramentas	Copiar
Item do menu de contexto	Copiar
Atalho Ctrl+C	Copiar

- Onde colocar a implementação da operação para que todos esses elementos possam utilizá-la?

Command - Solução Ingênua

- Criar subclasses de botões para implementar cada operação:
  - `BotaoCopiar`
  - `BotaoColar`
  - `BotaoRecortar`
- Colocar a lógica da operação dentro dessas subclasses
- Ao adicionar menus e atalhos, precisamos reutilizar essa lógica
- Isso pode levar a:
  - Duplicação da operação em diferentes elementos
  - Dependências entre menus, atalhos e botões
  - Alterações na interface gráfica sempre que a lógica da operação mudar

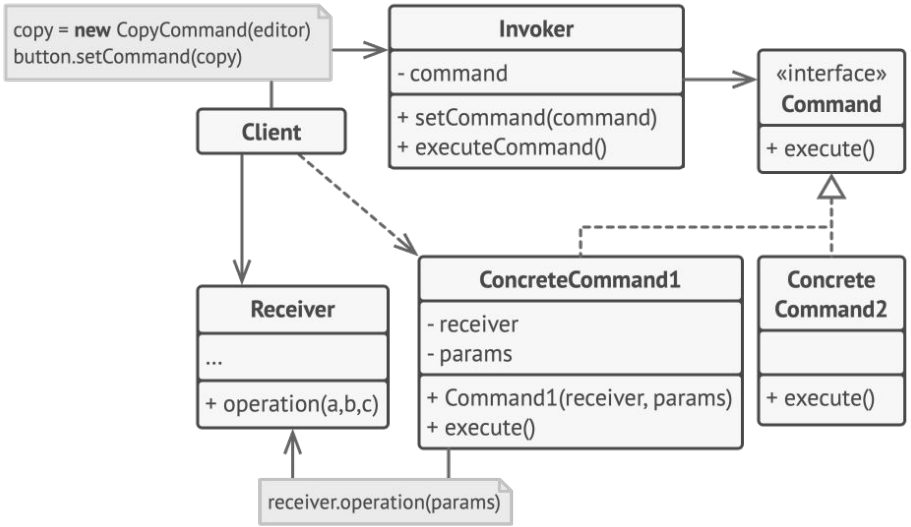
Command - Ideia e Intenção

"Encapsular uma solicitação em um objeto, reunindo as informações necessárias para executá-la"

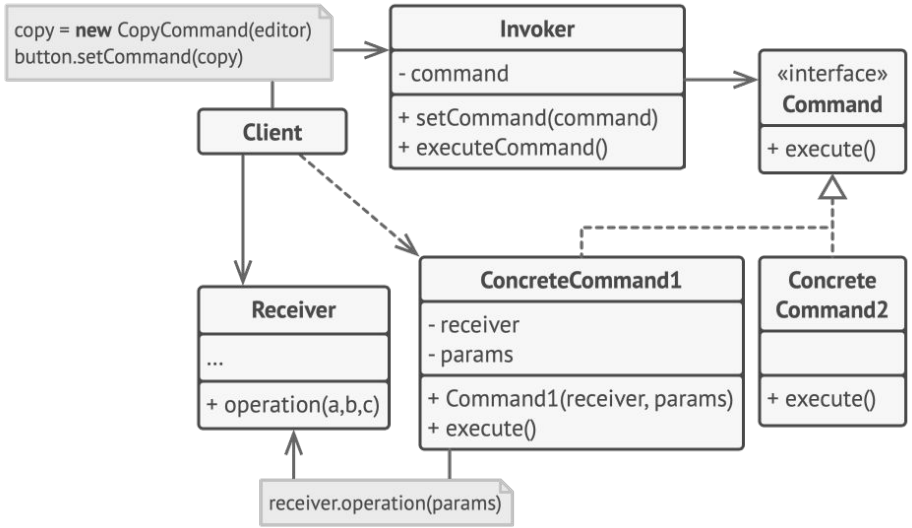
- No editor:
  - Um comando representa a solicitação de copiar
  - O comando conhece os objetos envolvidos na operação
  - Botão, menu e atalho acionam a mesma interface de execução
  - Os detalhes da operação ficam no comando e nos objetos aos quais ele delega o trabalho
- A solicitação passa a ter uma representação que pode ser armazenada, configurada e executada posteriormente

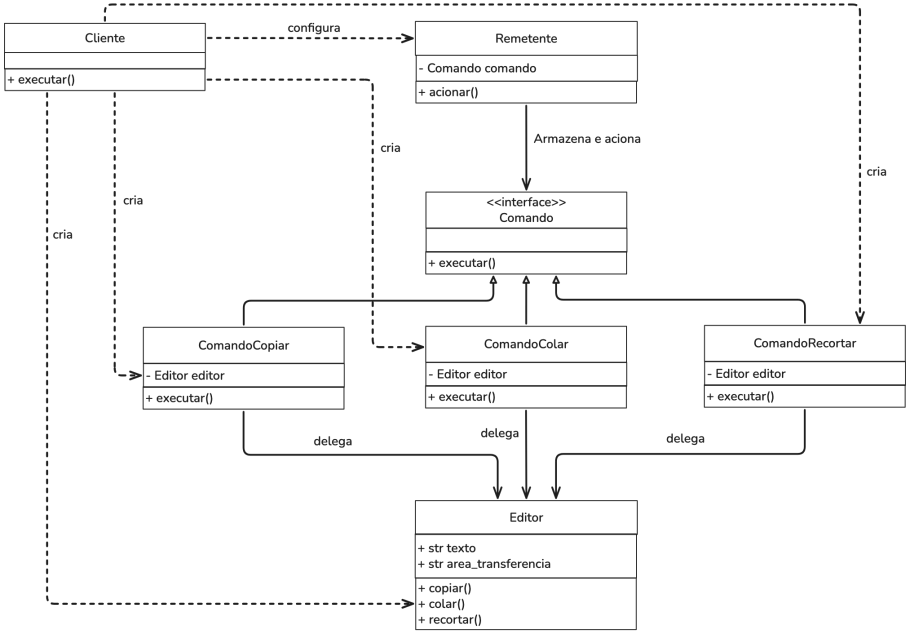
Command - Estrutura

1. **Invoker** mantém uma referência ao comando e solicita sua execução
2. **Command** declara a interface de execução
3. **ConcreteCommands** representam pedidos e mantêm as referências e os parâmetros necessários
4. **Receiver** implementa a lógica que realiza a operação
5. **Client** cria os objetos e configura suas relações



Aplicando ao nosso contexto





Aplicando ao nosso contexto

```python
class Editor:
    def __init__(self, texto: str) -> None:
        self.texto = texto
        self.area_transferencia = ""

    def copiar(self) -> None:
        self.area_transferencia = self.texto

    def colar(self) -> None:
        self.texto += self.area_transferencia

    def recortar(self) -> None:
        self.copiar()
        self.texto = ""
```

```python
from abc import ABC, abstractmethod


class Comando(ABC):
    @abstractmethod
    def executar(self) -> None:
        pass
```

```python
class ComandoCopiar(Comando):
    def __init__(self, editor: Editor) -> None:
        self._editor = editor

    def executar(self) -> None:
        self._editor.copiar()


class ComandoColar(Comando):
    def __init__(self, editor: Editor) -> None:
        self._editor = editor

    def executar(self) -> None:
        self._editor.colar()


class ComandoRecortar(Comando):
    def __init__(self, editor: Editor) -> None:
        self._editor = editor

    def executar(self) -> None:
        self._editor.recortar()
```

```python
class Remetente:
    def __init__(self, comando: Comando) -> None:
        self._comando = comando

    def acionar(self) -> None:
        self._comando.executar()
```

```python
class Cliente:
    @staticmethod
    def executar() -> None:
        editor = Editor("UPE")

        copiar = ComandoCopiar(editor)
        colar = ComandoColar(editor)
        recortar = ComandoRecortar(editor)

        botao = Remetente(copiar)
        menu = Remetente(copiar)
        atalho = Remetente(copiar)

        botao.acionar()
        menu.acionar()
        atalho.acionar()
        print("Área de transferência:", editor.area_transferencia)

        Remetente(colar).acionar()
        print("Após colar:", editor.texto)

        Remetente(recortar).acionar()
        print("Após recortar:", repr(editor.texto))
        print("Área de transferência:", editor.area_transferencia)
```

```python
Cliente.executar()
```

Prós e Contras do Command

Prós

- Separa os objetos que solicitam operações daqueles que realizam o trabalho
- Permite associar a mesma operação a diferentes remetentes
- Novos comandos podem implementar a interface sem alterar a classe Remetente
- Permite armazenar pedidos e adiar sua execução

Contras

- Acrescenta classes e uma camada de chamadas entre remetente e destinatária
- Pedidos simples podem exigir mais estrutura do que uma função ou chamada direta
- Desfazer e refazer exigem implementação específica, com histórico e informações para reverter as operações

Use:

- Operações acionadas por diferentes origens ou solicitações que precisam ser armazenadas, organizadas ou executadas posteriormente

Iterator - O Problema

- Considere uma coleção que recebeu estas palavras, nesta sequência:
  - `["banana", "abacaxi", "caju"]`
- Queremos oferecer dois percursos:
  - Ordem alfabética
    - `"abacaxi", "banana", "caju"`
  - Ordem alfabética inversa
    - `"caju", "banana", "abacaxi"`
- Também queremos permitir que dois percursos estejam em andamento ao mesmo tempo, cada um com sua própria posição
- Se o Cliente precisar conhecer o armazenamento e controlar todos esses detalhes, ficará acoplado à implementação da coleção
- O Iterator separa essas responsabilidades

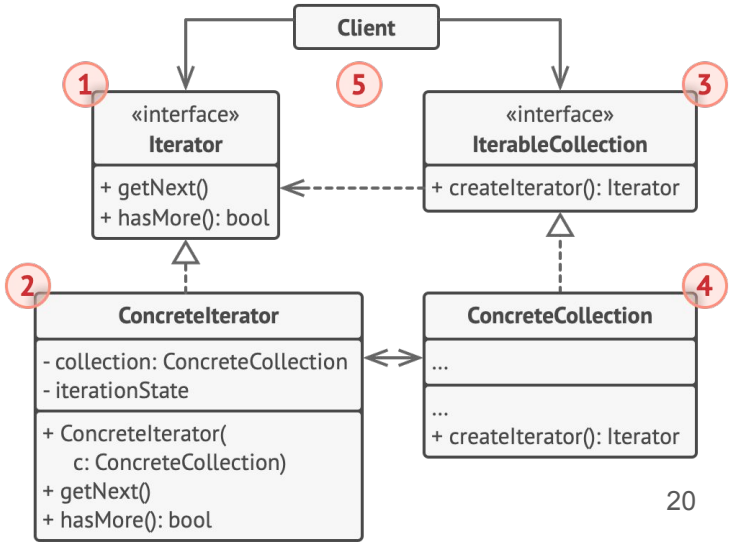
Iterator - ideia e intenção

"Permitir percorrer os elementos de uma coleção sem expor sua representação interna"

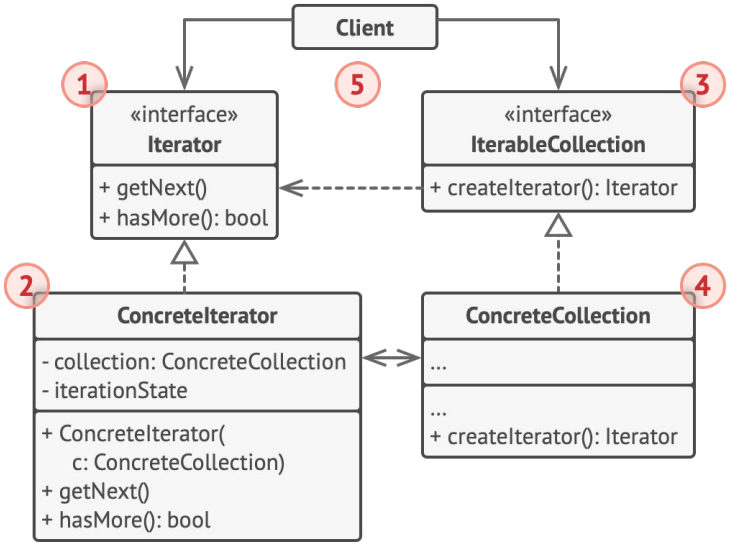
- Extrair o comportamento de percurso para um objeto separado: o Iterador
  - A Coleção armazena os elementos e fornece os iteradores
  - Cada Iterador controla sua posição e determina o próximo elemento
  - O Cliente solicita os elementos pela interface do Iterador
- Aplicando ao nosso contexto:
  - Um iterador percorre as palavras em ordem alfabética e outro realiza o percurso inverso
  - Cada iterador mantém seu próprio progresso, sem alterar a ordem de armazenamento das palavras

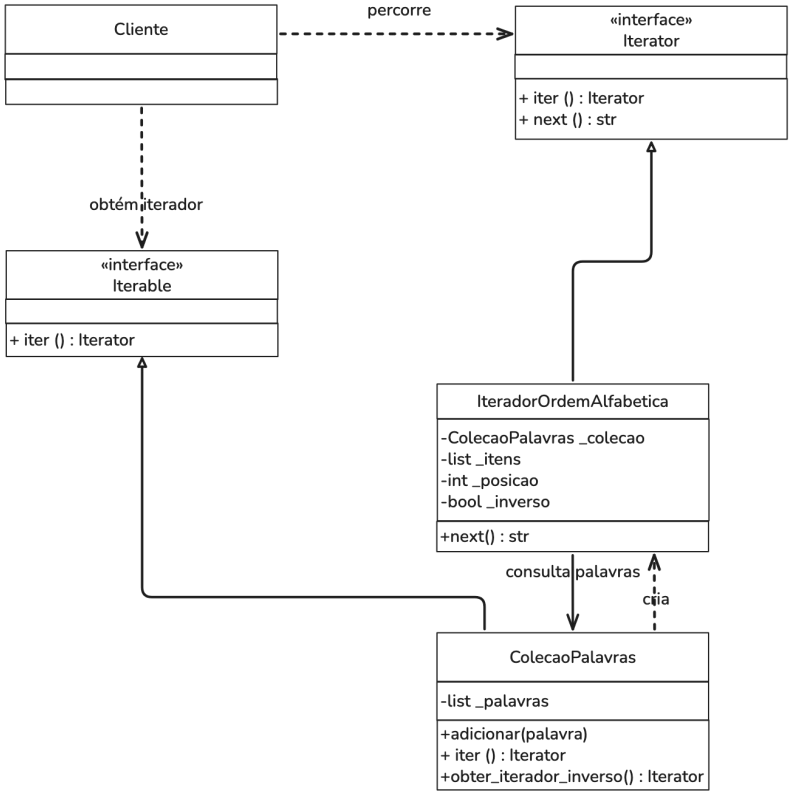
Iterator - Estrutura

1. A interface Iterador declara as operações necessárias para percorrer uma coleção
2. Iteradores Concretos implementam algoritmos específicos para percorrer uma coleção
3. A interface Coleção declara um ou mais métodos para obter os iteradores compatíveis com a coleção
4. Coleções Concretas retornam novas instâncias de uma classe iterador concreta em particular cada vez que o cliente pede por uma
5. O Cliente trabalha tanto com as coleções como os iteradores através de suas interfaces



Aplicando ao nosso contexto





Aplicando ao nosso contexto - Iterador Concreto

```python
from __future__ import annotations
from collections.abc import Iterable, Iterator


class IteradorOrdemAlfabetica(Iterator[str]):
    def __init__(
        self, colecao: ColecaoPalavras, inverso: bool = False
    ) -> None:
        self._colecao = colecao
        self._inverso = inverso
        self._itens: list[str] | None = None
        self._posicao = 0

    def __next__(self) -> str:
        if self._itens is None:
            self._itens = sorted(self._colecao._palavras, reverse=self._inverso)

        if self._posicao >= len(self._itens):
            raise StopIteration

        palavra = self._itens[self._posicao]
        self._posicao += 1
        return palavra
```

- A ordenação acontece na primeira solicitação de um elemento
- `sorted()` cria uma lista ordenada, preservando a ordem de armazenamento
- Cada instância mantém sua própria `_posicao`

Aplicando ao nosso contexto - Coleção Concreta

```python
class ColecaoPalavras(Iterable[str]):
    def __init__(self) -> None:
        self._palavras: list[str] = []

    def adicionar(self, palavra: str) -> None:
        self._palavras.append(palavra)

    def __iter__(self) -> Iterator[str]:
        return IteradorOrdemAlfabetica(self)

    def obter_iterador_inverso(self) -> Iterator[str]:
        return IteradorOrdemAlfabetica(self, inverso=True)
```

- `__iter__()` fornece o percurso alfabético
- `obter_iterador_inverso()` fornece o percurso alfabético inverso
- Cada chamada cria um novo iterador, com seu próprio progresso

Aplicando ao nosso contexto - Execução

```
colecao = ColecaoPalavras()
for palavra in ["banana", "abacaxi", "caju"]:
    colecao.adicionar(palavra)

print(list(colecao))
print(list(colecao.obter_iterador_inverso()))
```

- A coleção recebe as palavras na ordem de inserção
- Cada chamada solicita um novo percurso
- O iterador ordena uma cópia, preservando o armazenamento original

Aplicando ao nosso contexto - Dois iteradores

```
primeiro = iter(colecao)
segundo = iter(colecao)

print(next(primeiro))
print(next(primeiro))
print(next(segundo))

print(list(primeiro))
print(list(segundo))
```

- Avançar primeiro não modifica a posição de segundo
- `list(iterador)` consome os elementos restantes daquele percurso
- Cada iterador pode ser utilizado em momentos diferentes

Prós e contras do Iterator

Prós

- Separa o armazenamento dos dados da lógica de percurso
- Permite intercalar percursos independentes sobre a mesma coleção
- Oferece uma forma comum de consumir coleções diferentes

Contras

- Pode acrescentar estrutura desnecessária para coleções simples
- Pode apresentar custos adicionais em relação ao acesso direto aos elementos
- Exige implementar corretamente o controle e o encerramento do percurso

Use:

- Quando precisar percorrer uma estrutura complexa sem expor seus detalhes internos ao Cliente
- Quando vários trechos da aplicação repetirem a lógica de percurso
- Quando o mesmo Cliente precisar percorrer coleções diferentes por uma interface comum



Prof. Dr. Carlos Melo
https://calendly.com/prof-casm
carlos.melo@upe.br

UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS